IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 16 October 2025, accepted 22 November 2025, date of publication 26 November 2025,
date of current version 4 December 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3637569

RESEARCH ARTICLE

Analyzing LLM-Generated Code According to Four ISO/IEC 5055:2021 Categories

RASMUS KREBS AND SOMNATH MAZUMDAR, (Member, IEEE)
Department of Digitalization, Copenhagen Business School, 2000 Frederiksberg, Denmark
Corresponding author: Somnath Mazumdar (sma.digi@cbs.dk)

ABSTRACT The use of large language models (LLMs) for generating code in software development is on the rise. While LLMs demonstrate impressive capabilities, there remains a need to evaluate the quality of generated code beyond functional correctness. This paper addresses a gap in current research by evaluating Python code generated by nine state-of-the-art LLMs according to the four main code quality categories (maintainability, reliability, performance efficiency, and security) defined by the ISO/IEC 5055:2021 standard. The evaluation spans three popular application domains (high-performance computing, machine learning, and data processing). It employs a stratified prompting approach with varying levels (short, medium, and long) of detail. Using widely adopted static code analysis tools, the study identifies which LLMs perform best across these categories. The analysis involved selecting nine algorithms across three application domains, and the generated code was compared against a human developer using four static code analysis tools. Metrics were organized into the four ISO 5055 categories, and composite scores were calculated for each category following preprocessing to ensure accurate evaluation. The results showed that `GPT-4-Turbo` produced the most reliable, performance-efficient, and secure code, while `Gemini` excelled in generating maintainable Python code among the evaluated models. The study concludes that properly prompted and configured LLMs can produce code that meets or even exceeds human-developed code in terms of the four ISO categories. Future work will continue to refine and expand upon these methodologies on other programming languages.

INDEX TERMS Analysis, ISO, LLMs, maintainability, performance, Python, quality, reliability, security.

I. INTRODUCTION

Reports on the enterprise adoption of artificial intelligence (AI) show that 42% of large IT organizations have actively deployed AI, with an additional 40% reporting that they are actively exploring the technology [1]. In particular, specialized large language models (LLMs) for code generation are increasingly being used to enhance the productivity of software developers [2].

Existing research evaluating LLMs for coding reveals several prevalent trends, notably a strong emphasis on functional correctness, which has been a primary focus of current LLM research [3], [4], [5], [6], [7], [8], [9] and its application in real-world scenarios [2], [10], [11], [12], [13], [14], [15], [16]. However, concerns have been raised regarding the limited ability of such evaluations to capture real-world developer interactions. Consequently, several studies have called for more comprehensive and context-aware assessments [3], [5], [12], [17].

A review of the current literature reveals a notable gap: no prior studies have comprehensively evaluated LLM-generated code across all four ISO/IEC 5055:2021 (ISO 5055) quality categories (such as maintainability, reliability, performance efficiency, and security) [18]. Research continues to show limited attention to performance efficiency [3] and reliability [4], despite their critical importance to software development [19]. Additionally, LLMs often under-perform in real-world applications compared to controlled environments, underscoring the need for broader, more representative evaluations and improved integration of LLMs into software engineering processes [20], [21].

The associate editor coordinating the review of this manuscript and approving it for publication was Fabrizio Messina.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

Motivated by these gaps, this study evaluates the quality of LLM-generated Python code (similar to [22], [23]) across the four principal ISO 5055 categories. For this purpose, nine popular and high-ranking conversational LLMs were selected based on the EvalPlus leaderboard.[1] Nine evaluated LLMs are: `GPT-4-Turbo` (GPT-4T), `DeepSeek-Coder-33B-Instruct` (DeepSeekCoder), `OpenCodeInterpreter-DS-33B` (OpenCodeInterpreter), `GPT-3.5-Turbo` (GPT-3.5), `WizardCoder-33B-V1.1` (WizardCoder), `Magicoder-S-DS6.7B` (Magicoder), Codex, `Gemini Pro 1.0` (Gemini), and `CodeLLaMA70B-Instruct` (CodeLLaMA).

This research is guided by the hypothesis that LLMs can be optimized to generate high-quality Python code that adheres to established industry standards. Our primary research question is defined as follows: **To what extent can LLMs conform to the four principal categories of ISO 5055?** The study aims to determine which LLMs perform best within these categories, thereby providing Python developers with evidence-based insights for selecting models that align with their software quality requirements.

A. CONTRIBUTION

This research contributes to the ongoing integration of LLMs in code generation. Through a comprehensive analysis, the following key findings were identified:

1) **Maintainability:** Based on the Maintainability Index, `Gemini` achieved the highest overall score, closely followed by `GPT-4T`. Interestingly, models that performed poorly in memory usage often exhibited stronger maintainability, suggesting a trade-off between resource efficiency and structured code organization.
2) **Performance Efficiency and Security:** `GPT-4T` produced the most reliable and performance-efficient Python code while also demonstrating superior security compared to the other eight evaluated models.
3) **Prompt Sensitivity:** Prompt length significantly influenced code reliability and error rates. However, model-specific characteristics also played a critical role in determining overall performance, indicating that prompt optimization alone cannot ensure consistent quality.
4) **Prompt Design Insights:** Medium-length prompts generally produced more reliable and secure code, while longer prompts yielded the highest adherence to Python-specific metrics and coding conventions.

The above findings not only highlight model-specific strengths and trade-offs but also offer practical guidance for developers on selecting and configuring LLMs to generate high-quality, standards compliant software.

[1] https://evalplus.github.io/leaderboard.html

II. BACKGROUND

ISO 5055 is a key standard jointly issued by the International Organization for Standardization (ISO) and the International Electrotechnical Commission (IEC). It provides comprehensive guidelines for assessing violations of good architectural and coding practices in source code, thereby enabling organizations to mitigate operational risks and reduce associated costs. Advancements in measuring software structural quality and detecting coding practice violations through language-specific static analysis tools have highlighted the need for a more robust and unified framework [18]. Consequently, ISO 5055 was developed to address this gap by offering a standardized methodology that enhances the precision, reliability, and consistency of code quality evaluation across multiple programming languages and development contexts.

The standard defines four principal categories for evaluating software quality:

- **Maintainability:** The ability of source code to be modified efficiently and effectively, allowing developers to implement changes within a reasonable time frame.
- **Performance Efficiency:** The extent to which code utilizes computational resources appropriately; excessive resource consumption can negatively impact other software processes.
- **Reliability:** The capability of code to perform consistently under specified and intended conditions, ensuring that it executes as designed.
- **Security:** The ability of code to safeguard information and data from unauthorized access, misuse, or interference by external systems or users.

As ISO 5055 is relatively new, only a limited number of static analysis tools fully support it, particularly for Python. However, widely used open-source tools can be grouped according to the ISO 5055 metrics, enabling practical and automated assessments. Aligning this study with ISO 5055 facilitates the use of such tools to systematically evaluate LLM-generated code across maintainability, performance efficiency, reliability, and security providing a comprehensive and standardized assessment of LLMs' ability to generate high-quality Python code.

III. RELATED WORK

LLM-based code generation has emerged as a prominent research area, and the selected studies in this work are categorized according to the ISO 5055 quality categories.

A. MAINTAINABILITY

Maintainability is the most frequently examined ISO 5055 category in the literature. Molison et al. [23] evaluated `GPT-4o`-generated Python code of varying difficulty levels using SonarQube across three model configurations (i.e., zero-shot, few-shot, and fine-tuned). Their analysis focused on maintainability and reliability, revealing that LLM-generated code contained fewer bugs overall.

Poldrack et al. [2] reported that GPT-4T effectively simplifies and standardizes source code, improving maintainability through realistic developer interaction simulations. In contrast, Vaithilingam et al. [15] found that programmers often struggled to understand and modify generated code, highlighting usability challenges and maintainability concerns. Similarly, Bubeck et al. [10] observed that GPT-4T generated code can be difficult to maintain due to frequent syntactic and semantic errors. Mastropaolo et al. [11] examined the impact of prompting on code generation with Codex and found that varying natural language descriptions produced different outputs in 46% of cases, even when functional requirements were identical.

Although these studies report both maintainable [2] and less maintainable [10], [11], [15] LLM-generated code, none have systematically evaluated maintainability across all ISO 5055 dimensions, indicating an ongoing gap in comprehensive code quality assessments.

B. PERFORMANCE EFFICIENCY

Performance efficiency remains one of the most overlooked categories within ISO 5055. Nascimento et al. [12] found that while ChatGPT can outperform novice programmers, it struggles to match the performance of more experienced developers. Similarly, Huang et al. [3] reported that GPT-4T generates the most computationally efficient code among the evaluated models. However, research focused on performance efficiency remains limited and often relies on similar benchmark-based methodologies [3], [12]. A key threat to the validity of such studies is the potential inclusion of LeetCode[2] tasks within LLM training datasets. These widely available problems are frequently used for instruction tuning, making it plausible that models have been indirectly exposed to them during training. This overlap underscores the need for alternative evaluation approaches that assess performance efficiency on independent and diverse codebases.

C. RELIABILITY

Reliability is another frequently examined ISO 5055 category in LLM research [2], [10], [11], [15]. A common focus across these studies is the evaluation of functional correctness using standardized benchmarking datasets. Among these, HumanEval [8] and the Mostly Basic Programming Problems (MBPP) benchmark [9] are widely used to assess language comprehension, algorithmic reasoning, and basic mathematical capabilities of generated code. However, these benchmarks suffer from limited test coverage and vague problem descriptions [5]. Chen et al. [8] identified reliability limitations in Codex, particularly when handling complex operation chains. Most existing benchmarks emphasize function-specific problem descriptions, a limitation that Du et al. [6] addressed through the introduction of ClassEval, a benchmark designed for class-level code generation. Their findings revealed that LLMs perform significantly worse when generating full class implementations compared to isolated functions. These results, along with prior research [5], [8], [9], highlight the shortcomings of functionality-focused benchmarks and raise concerns about the overall reliability of LLM-generated code. Li et al. [7] proposed an alternative benchmark for evaluating code review tasks, while Zhong and Wang [4] introduced a reliability-oriented benchmark based on 1,208 Stack Overflow programming questions. Their study revealed recurring patterns of unreliability and potential security issues across models. Overall, most reliability research remains centered on functional correctness, which, while useful for model improvement, does not fully capture real-world software dependability. Schäfer et al. [13] approached reliability from a more applied perspective, demonstrating that an adaptive GPT-3.5T-based test generation tool can outperform existing state-of-the-art methods. Nevertheless, only one identified study explicitly addresses reliability as a primary research objective, emphasizing the need for more comprehensive, empirically grounded evaluations of LLM reliability in realistic development contexts.

D. SECURITY

Although security is one of the most frequently cited concerns regarding LLM-generated code, research providing a thorough evaluation of these issues remains limited. Licorish et al. [22] compared GPT-4 and human programmers in generating Python software, focusing on code security and reliability. Their results showed that while GPT-4 can produce functionally effective solutions, it often generates more complex code that requires additional reworking to ensure maintainability. Pearce et al. [16] explored the security implications of Codex by prompting it to generate code for high-risk cybersecurity scenarios and found that approximately 40% of the 1,689 programs produced contained vulnerabilities listed in the Common Weakness Enumeration (CWE). Similarly, Perry et al. [14] observed that users with access to LLMs wrote significantly less

TABLE 1. Comparing works against maintainability (M), performance efficiency (PE), reliability (R), and security (S).

Paper	M	PE	R	S	LLMs
Huang et al. [3]	-	×	×	-	21 LLMs
Zhong et al. [4]	-	-	×	×	GPT's, CodeLLama 2, DeepSeek-Coder
Poldrack et al. [2]	×	-	×	-	GPT-4
Vaithilingam et al. [15]	×	-	×	-	Codex
Mastropaolo et al. [11]	×	-	×	-	Codex
Bubeck et al. [10]	×	-	×	-	GPT-4
Nascimento et al. [12]	-	×	-	-	ChatGPT
Liu et al. [5]	-	-	×	-	26 LLMs
Du et al. [6]	-	-	×	-	11 LLMs
Schäfer et al. [13]	-	-	×	-	GPT-3.5-Turbo
Perry et al. [14]	-	-	×	×	Codex
Li et al. [7]	×	-	×	-	CodeReviewer
Austin et al. [9]	-	-	×	-	Undisclosed
Pearce et al. [16]	-	-	×	×	Codex
Chen et al. [8]	×	-	×	×	Codex
Molison et al. [23]	×	-	×	-	GPT-4o
Licorish et al. [22]	-	-	×	×	GPT-4

[2]https://leetcode.com/problemset/

secure code than those without, suggesting that reliance on LLM-generated output may lead to overconfidence and insufficient verification. This finding aligns with earlier observations by Chen et al. [8], who emphasized the need for human review of LLM-generated code prior to deployment. Moreover, Zhong and Wang [4] found that `GPT-4T`-generated code exhibited API misuse in 62% of analyzed cases, potentially leading to unintended or insecure behaviors in production environments.

Table 1 compares related studies across the four categories. As shown, most prior research evaluates code generated by only a single LLM, thereby limiting the generalizability of their findings. Chen et al. [8] addressed more than two of the ISO 5055 categories but did not explore them in depth, focusing primarily on the broader implications of LLMs such as `Codex`. Wang et al. [20] argued that LLMs perform significantly less efficiently in real-world contexts compared to controlled benchmarking environments, emphasizing the need for more comprehensive empirical evaluations. Similarly, Fan et al. [21] identified persistent challenges related to the maintenance and effective integration of LLMs within software engineering workflows.

IV. METHODOLOGY

Code synthesis has been a long-standing challenge in computer science [9], [24], [25], [26]. The advent of attention mechanisms and transformer architectures [27] enabled scalable sequence prediction and efficient natural language processing (NLP), forming the foundation of modern LLMs [25]. Subsequent research demonstrated that transformer-based LLMs exhibit strong cross-domain capabilities derived from large-scale training data, particularly excelling in understanding and generating source code [28], [29]. Consequently, transformer-based and code-specialized LLMs have become central to contemporary programming code synthesis research.

Source code analysis provides developers with insights into potential issues affecting maintainability, performance efficiency, reliability, and security [30]. Python, being dynamically typed, presents unique challenges related to runtime type-checking. To address this, Python organizations established style and documentation guidelines i.e., *Python Enhancement Proposals*, notably *PEP 8* and *PEP 257* [31]. As these standards are not enforced by default, open-source static analysis tools have been developed to automate compliance, supporting higher code quality. This study examines how different LLMs interpret and execute code generation tasks under varying prompt specificity and contextual detail. A quantitative methodology is employed to assess the extent to which LLM-generated code aligns with ISO/IEC 5055:2021 standards. The overall research process is illustrated in Figure 1.

A. LLM SELECTION

ISO/IEC 5055:2021 provides a standardized framework for assessing the internal structure of software with emphasis on security, reliability, performance efficiency, and maintainability [18]. To evaluate functional correctness in code generation, benchmarks such as *HumanEval* [8] and its popular alternative *MBPP* [9] are widely used. The *EvalPlus* leaderboard served as the primary source for model selection, offering quantitative assessments of LLM performance across enhanced versions of these benchmarks, using both individual and averaged scores.

The benchmark employs the *pass@1* metric with greedy decoding [32], where *pass@1* measures the proportion of unit tests passed by the first generated solution [8], and greedy decoding iteratively selects the most probable next token [33].

Smaller models, including `OpenCodeInterpreter-DS` (ranked 8) and `DeepSeek-Coder-6.7B-Instruct` (ranked 9), were excluded in favour of their larger, better-performing counterparts. In addition to EvalPlus-listed models, three widely adopted LLMs such as `Codex`, `Gemini Pro 1.0`, and `CodeLLaMA-70B-Instruct` were included to ensure diversity across architectures and training paradigms. The final selection (`GPT-4T`, `DeepSeek-Coder`, `OpenCodeInterpreter`, `GPT-3.5T`, `WizardCoder`, `Magicoder`, `Codex`, `Gemini`, and `CodeLLaMA`) thus represents a balanced mix of top-performing and widely deployed models for a comprehensive comparative evaluation.

B. APPLICATION DOMAIN

Nine representative algorithms spanning three key application domains (such as high-performance computing (HPC), machine learning (ML), and data processing) were selected to capture the diversity of real-world software tasks.

1) HPC

The chosen algorithms, *Quick Sort (QS)*, *Strassen Matrix Multiplication (SMM)*, and *Monte Carlo Simulation (MCS)*, represent computationally intensive operations. MCS, used in financial modeling, tests the LLMs' ability to handle probabilistic reasoning and large-scale iterations.

2) ML

This domain includes *Attention*, *Convolution*, and *Principal Component Analysis (PCA)*. The *Attention* mechanism [27] central to modern NLP and the foundation of LLM architectures requires managing complex dependencies and efficient computation. *Convolution* is fundamental in image processing and computer vision, emphasizing spatial and numerical precision, while *PCA* reduces data dimensionality by identifying key components explaining the greatest variance.

3) DATA PROCESSING

Algorithms in this category focus on transforming raw data into usable forms. The selected algorithms are *Huffman Coding (HC)* for compression, *PageRank (PR)* for graph-based data analysis, and *Rabin–Karp (RK)* for efficient string

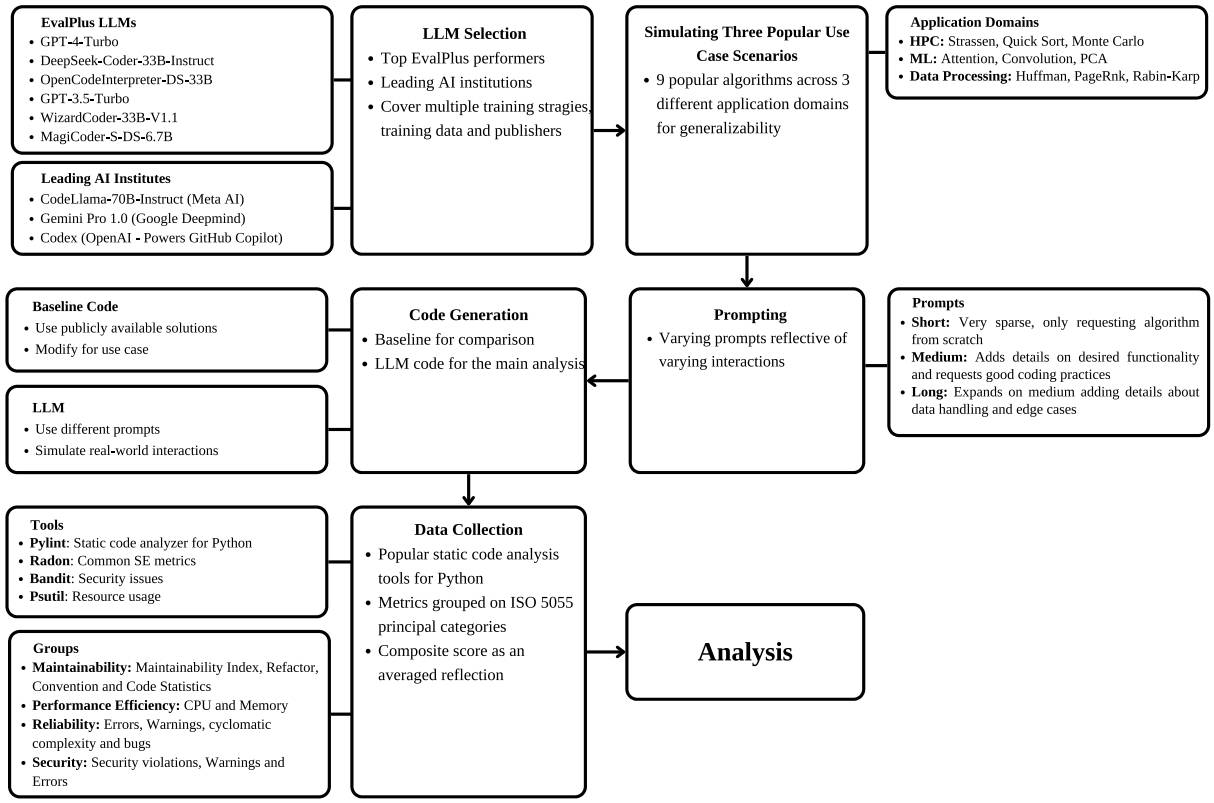


FIGURE 1. Applied research methodology for code analysis.

```
Implement convolution from scratch in python, taking padding, stride and dialation as inputs. In a
run pass, the image and kernel is passed as inputs.
```

FIGURE 2. Example of short prompt for convolution code generation.

```
Implement convolution from scratch in python. At initalization it should take the padding, stride
and dialation. The run method should take an image and a  kernel as inputs, and  return the
convolved image. Make sure to add useful  methods and steps to handle common edge cases for the
image and kernel sizes.

Make sure to add all the necessary documentation to the class and methods in the form of docstrings
and comments. Also make sure to add type hints throughout.
```

FIGURE 3. Example of medium prompt for convolution code generation.

pattern matching. Together, these domains and algorithms ensure a comprehensive and domain-balanced evaluation of LLM performance across computational, analytical, and data-centric problem spaces.

C. PROMPTING

Prompting, the process of designing and refining input queries to guide LLM outputs, is referred to as *prompt engineering* [34]. Common techniques include *role-play prompting* [35] and the *chain-of-thought* method [36], both aimed at improving reasoning depth and contextual relevance. To ensure the generalizability of results, this study adopts a stratified prompting approach classified by prompt length into short, medium, and long (based on the methodology proposed by [11]).

1) SHORT PROMPT

Short prompts request the implementation of an algorithm from scratch with minimal context, relying on the LLM to infer requirements and apply general coding best practices. This setup evaluates the model's ability to abstract and generalize beyond explicit instructions. However, limited contextual information may introduce ambiguity, often leading to incomplete or incorrect algorithmic implementations (see Figure 2).

2) MEDIUM PROMPT

Medium prompts specify functionality, use cases, documentation, and type hints but provide limited details on argument handling or data types (see Figure 3). This format

```
Implement convolution from scratch in Python. The class should at initalization take the padding,
stride and dialation as inputs, in either integer or a tuple  corresponding to width and height.
Implement the necessary steps to handle these  differing inputs, such as transforming integers into
a tuple with each value  equaling the input integer. Moreover, check that each input corresponds to
an  interger. After the class is initialized, the run method should take an image  and a kernel as
inputs. Add methods to handle common edge cases such as the  kernel not being a square matrix, that
the kernel is the correct size for the  image. If the kernel is a list, it should be converted to a
numpy array.

Make  sure to check that the kernel is a square matrix and that it is the correct  size for the
image. The output should be the convolved image.   Make sure to add all the necessary documentation
to the class and methods in the  form of docstrings and comments. Also make sure to add type hints
throughout. Follow the common conventions for Python code
```

FIGURE 4. Example of long prompt for convolution code generation.

encourages the LLM to engage in creative problem-solving and interpretive reasoning, fostering the generation of more adaptable and context-aware solutions.

3) LONG PROMPT

Long prompts expand upon medium prompts by adding explicit instructions for handling edge cases, undefined arguments, and specific data types (see Figure 4). Designed to minimize ambiguity, this structure guides the LLM toward optimal solutions through detailed specification, thereby testing its ability to process complex, instruction rich scenarios with precision.

The stratification of prompt lengths serves two primary research objectives. First, it assesses the adaptability of LLMs to varying levels of instruction detail, enabling evaluation across diverse coding scenarios. Second, it provides insights into model-specific biases, adherence to coding conventions, and the capacity to exceed prompt constraints in pursuit of higher-quality solutions. This approach also reflects the iterative nature of software development, where initial drafts are refined through successive interactions, allowing assessment of each model's consistency and refinement capability.

If an LLM produced an unsatisfactory output, follow-up prompts were issued without adding new information to maintain experimental consistency. All models were prompted independently, ensuring no contextual carryover between iterations. While varying prompt lengths naturally lead to differences in usability and output quality, longer prompts are generally expected to yield more functionally complete solutions. Nonetheless, each LLM is expected to produce code that meets baseline quality standards regardless of prompt complexity. This stratified design also captures differences in user intent and developer behavior, illustrating how prompt specificity influences both performance and generalization.

D. CODE GENERATION

LLM-generated code was obtained by querying each model using the prompts defined in IV-C. For `GPT-4T` and `GPT-3.5T`, inference parameters (such as maximum output length and temperature) were available but kept at default values, except that the maximum length was increased to prevent premature truncation. Models including `OpenCodeInterpreter`, `WizardCoder`, `MagiCoder`, and `CodeLLaMA` lack dedicated graphical interfaces and were instead executed locally via Ollama,[3] a command-line tool for running LLMs. `Codex` was accessed through GitHub Copilot within Visual Studio Code editor, using its integrated Copilot panel for code generation.

E. BASELINE

A human-developer baseline was established to provide a consistent benchmark for evaluating how LLMs compare to human-generated Python code. The baseline solutions were developed by the first author, who possesses over three years of professional experience in Python programming and adheres to standard coding conventions outlined in PEP 8 and PEP 257. This single-developer baseline was intentionally chosen to ensure experimental consistency and eliminate stylistic or methodological variability that could arise from involving multiple programmers. Each baseline solution was carefully written to reflect best practices and domain expertise across all selected application areas. To validate the selected metrics and ensure a fair comparison, all developer assistance tools and static analyzers were disabled during baseline creation. This approach ensures that the human benchmark provides a controlled, representative, and unbiased reference for assessing LLM-generated code quality across four ISO/IEC 5055:2021 categories.

F. DATA COLLECTION

Four open-source tools were employed in this study: `Pylint` (version 2.16.2) [37], `Radon` (version 5.1.0),[4] `Bandit` (version 1.7.6),[5] and Python System and Process Utilities (`psutil`, version 5.9.0).[6]

1) PYLINT

`Pylint` is a static Python code analysis tool that detects programmatic and stylistic errors, enforces coding standards, and identifies poor-quality code [38]. It classifies

[3] https://github.com/ollama/ollama
[4] https://radon.readthedocs.io/en/latest/
[5] https://bandit.readthedocs.io/en/latest/
[6] https://psutil.readthedocs.io/en/latest/

issues into five categories: *(i) Style Convention Errors (C)* violations of coding style guidelines defined in *PEP 8*; *(ii) Refactoring Checks (R)* recommendations for improving code structure and readability; *(iii) Warnings (W)* indicators of potentially problematic code segments; *(iv) Errors (E)* issues likely to cause runtime failures; and *(v) Fatal Errors (F)* severe issues that halt analysis. For this study, two style-related messages `line-too-long (C0301)` and `trailing-whitespace (C0303)` were disabled to prevent formatting discrepancies caused by user interface constraints.

2) RADON

`Radon` computes four key software metrics: *(i) Cyclomatic Complexity* measures the number of decision points in the code [39]; *(ii) Maintainability Index* evaluates code quality based on complexity, size, and documentation [40]; *(iii) Number of Delivered Bugs* estimates potential reliability issues using software complexity metrics [41]; and *(iv) Source Code Statistics* provides metrics such as lines of code (LOC) and the comments-to-code ratio.

3) BANDIT

It is a static analysis tool for Python that identifies potential security vulnerabilities. It scans code using multiple plugins to detect common security issues such as SQL injection risks, hard-coded passwords or keys, and the misuse of unsafe functions. Although the algorithms analyzed in this study do not inherently emphasize security, `Bandit` was included to identify any security-related or style-adjacent violations that could compromise code integrity.

4) PSUTIL

`psutil` is a system monitoring tool that provides detailed information on resource utilization, including CPU, memory, disk activity, and network interfaces. In this study, only CPU and memory usage data were collected. Monitoring with `psutil` required executing each solution to observe real-time system performance.

Any critical execution errors were addressed through follow-up prompts and minor adjustments to ensure that the LLM-generated code was executable. These adjustments included adding missing library imports, initializing variables within class attributes, and correcting matrix transpositions. If substantial modifications were necessary, the LLM was prompted to generate an alternative solution. It is important to note that this process did not confirm the functional correctness of the algorithms. It could still contain logical or implementation errors, while the code was executable.

5) SELECTED METRICES

Metrics obtained from the analysis tools were organized according to the ISO 5055 quality categories (see Table 2). Python-specific metrics included `Pylint` style and refactoring messages, the comments-to-code ratio, and source lines of code (SLOC) per method. Reliability was evaluated using Cyclomatic Complexity and the number of delivered bugs, while `Pylint` and `Bandit` contributed to the security assessment by identifying common security violations.

TABLE 2. ISO 5055 focused all metrics used for analysis.

ISO 5055	Metric	Tool
Maintability	Maintainability Index (MI)	Radon
	Convention (C)	Pylint
	Refactor (R)	Pylint
	Comments-to-LOC (C-to-LOC)	Radon
	SLOC-to-Methods (SLOC-to-M)	Radon
Performance Efficiency	CPU Usage (CPU)	psutil
	Memory Usage (Memory)	psutil
Reliability	Cyclomatic Complexity (CC)	Radon
	Delivered Bugs (Bugs)	Radon
	Warnings	Pylint
	Errors	Pylint
Security	Security Violations (SV)	Bandit
	Warnings	Pylint
	Errors	Pylint

G. COMPOSITE SCORE

Composite scores were calculated as weighted sums of the input metrics for each of the four ISO 5055 categories. These scores provide an averaged representation of each LLM's performance within a given category, balancing its relative strengths and weaknesses.

1) MAINTAINABILITY (M)

The maintainability score was divided into three components: the maintainability index, Python-specific maintainability metrics, and source code statistics (see Equation 1). Each component contributed one-third of the final weight. For sections containing multiple metrics (such as Python-related maintainability violations) the values were first averaged before inclusion in the composite score.

$$\mathrm{M} = \frac{1}{3}(\mathrm{MI} + \frac{C+R}{2} + \frac{\text{C-to-LOC} + \text{SLOC-to-M}}{2}) \quad (1)$$

It is important to note that styling preferences may vary across projects that adhere to different coding standards.

2) PERFORMANCE EFFICIENCY (PE)

This category comprises two equally weighted metrics, as defined in Equation 2. The metrics (CPU and memory utilization) represent the percentage of CPU and RAM resources consumed, respectively.

$$\mathrm{PE} = \frac{1}{2}(\mathrm{CPU} + \mathrm{Memory}) \quad (2)$$

3) RELIABILITY (R)

This category comprises two components, each containing two metrics. All four metrics were assigned equal weights, each contributing one-fourth of the total reliability score (see Equation 3).

$$\mathrm{R} = \frac{1}{4}(\mathrm{CC} + \mathrm{Bugs} + \mathrm{Warnings} + \mathrm{Errors}) \quad (3)$$

4) SECURITY (S)

While system reliability is essential for safety, it alone is not sufficient. Code safety contributes to reliability, and security serves as a means to achieve it [42]. In this analysis, security was evaluated using three metrics (Security Violations (SVs), Warnings, and Errors) all of which relate to code style. A composite score, assigning equal weight to each metric, was used for the final assessment (see Equation 4).

$$S = \frac{1}{3}(SVs + Warnings + Errors) \tag{4}$$

5) NOTE ON WEIGHT SELECTION

Although some metrics may hold greater relevance in specific contexts, equal weights were assigned across all composite scores in this study to ensure a consistent and unbiased evaluation framework. This approach minimizes subjectivity and enables objective comparison across metrics, thereby enhancing the generalizability and reproducibility of the results.

V. DATA CLEANING AND PREPROCESSING

Variations in prompt length and detail can lead to differences in code output and potential style violations, complicating direct comparisons. Longer prompts may introduce additional coding errors, thereby skewing the analysis and leading to biased assessments. To mitigate these effects, preprocessing was applied to normalize results across varying code lengths, ensuring fairness and comparability. The objective of this preprocessing step was to provide a clear, context-aware understanding of the data that, when aggregated with algorithms within the same domain, remains unbiased toward any specific code sample. Table 3 summarizes the preprocessing procedures applied to each metric.

A. SMOOTHING

To address zero-counting errors, *add-k* smoothing was applied. It is a standard technique used to stabilize sparse data distributions [43]. The parameter k was set to 0.01, which prevents unjust penalization of solutions while maintaining evaluation stability.

B. NORMALIZATION

Normalization (see Equation 5) was applied to count-based code violations by dividing each raw count by the total number of lines of code in the corresponding solution. This transformation expresses violations per line of code, ensuring consistent comparability across different prompt lengths.

$$\text{Normalized Violations} = \frac{\text{Number of Violations}}{\text{Total Lines of Code}} \tag{5}$$

The SLOC and Cyclomatic Complexity (CC) are normalized by using a number of methods. This approach assesses how well logic and structural complexity are dispersed across methods, rather than just lines of code.

TABLE 3. Summarizing each metrics of preprocessing (Max Observed (MO)).

Metric	Smoothing	Norm.	Scaling	Inverse
Maintainability Index	-	-	10	False
Convention	0.01	LOC	MO	True
Refactoring	0.01	LOC	MO	True
Comments	-	LOC	MO	False
SLOC	-	Methods	MO	True
CPU Usage	-	-	100	True
Memory Usage	-	-	MO	True
Cyclomatic Complexity	-	Methods	MO	True
Delivered Bugs	-	LOC	MO	True
Warnings	0.01	LOC	MO	True
Errors	0.01	LOC	MO	True
Security Violations	0.01	-	MO	True

C. SCALING

Scaling was applied to standardize all metrics for improved interpretability and cross-metric comparison (see Equation 6, where x_a represents the vector of all observed values for a given metric associated with algorithm a). Each metric was scaled to a range between zero and one, converting absolute measurements into relative values.

$$\text{Scaled Values} = \frac{x_a}{\max(x_a)} \tag{6}$$

For fixed-range metrics, such as the Maintainability Index (ranging from 0 to 100), the upper bound was used for scaling. Metrics in which higher values indicate lower performance were inverted by subtracting each value from one, ensuring that values closer to one uniformly represent better performance.

VI. RESULTS

A. ENVIRONMENT SETUP

An Apple MacBook Pro with an M2 Pro chip and 16 GB of RAM running macOS (Sonoma version 14.5) was used for the evaluation. The system was configured with Python v3.11, NumPy v1.24.4, SciPy v1.11.1, and Pandas v1.5.3. Baseline solutions were developed using the Visual Studio Code editor. Owing to the substantial computational demands of the evaluated LLMs, a virtual machine equipped with an NVIDIA A40 GPU and 40 GB of RAM was also utilized. Each LLM was prompted to implement every algorithm three times (corresponding to the different prompt lengths) resulting in a total of 243 Python code samples. To minimize the impact of external factors on performance metrics, each solution was executed five times in a controlled looping process.

B. MAINTAINABILITY

The analysis first evaluates the Maintainability Index, followed by Python-specific maintainability metrics based on *PEP 8* style conventions and refactoring checks, and finally

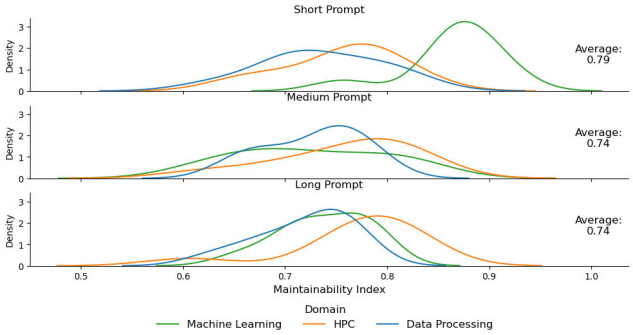


FIGURE 5. Maintainability index with varying prompts and domains.

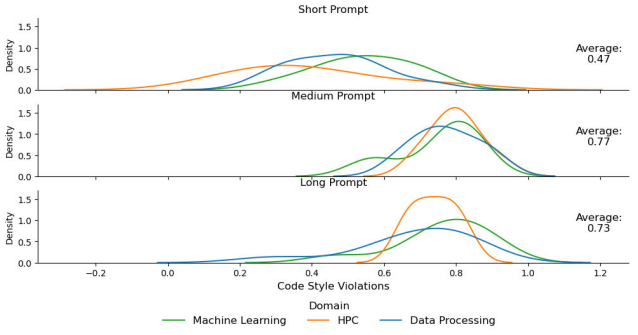


FIGURE 7. Style convention violations with varying prompts and domains.

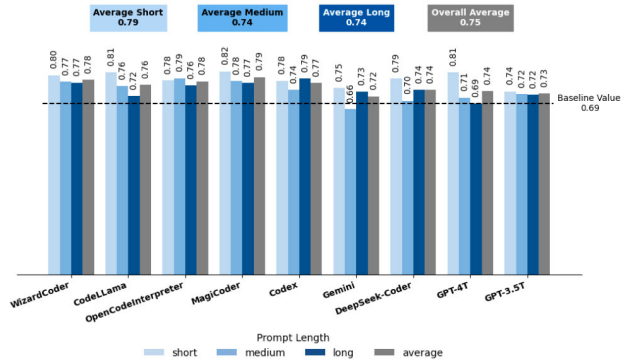


FIGURE 6. Maintainability index score across prompts.

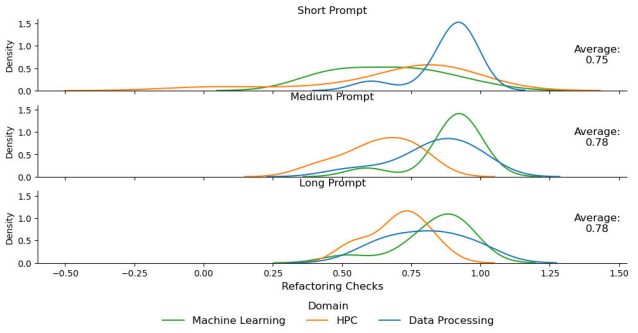


FIGURE 8. Refactor messages with varying prompts and domains.

assesses the comments-to-LOC ratio and SLOC-to-methods metrics.

1) MAINTAINABILITY INDEX

The distribution of the Maintainability Index across domains (see Figure 5) indicates that LLMs generally exhibit consistent performance, with relatively uniform dispersion within each domain. A notable exception occurs in the ML domain, where short-prompt solutions outperform longer ones. This outcome suggests that shorter prompts may lead to simpler implementations with lower code density and reduced complexity. However, this trend is not observed in other domains, indicating a possible domain-specific characteristic of ML-related tasks.

The Maintainability Index (see Figure 6) remains relatively consistent across the nine evaluated LLMs. `Magicoder`, `WizardCoder`, and `OpenCodeInterpreter` stand out slightly, performing approximately 10–11% above the baseline. Although none of the models exhibit substantial underperformance, `Gemini` records the lowest scores, particularly for medium-prompt solutions. Overall, all LLMs outperform the baseline, indicating that they are generally well-suited to balance code density, comment ratio, and complexity.

2) PYTHON CODING VIOLATIONS

Figure 7 presents the scores for style convention violations across the evaluated LLMs. Overall, the models perform notably worse in code convention adherence for short-prompt solutions, exhibiting greater score dispersion and a lower average. This variation decreases for medium and long prompts, where the distributions appear more compact. These results suggest that explicitly specifying adherence to Python coding conventions within the prompts may have positively influenced compliance. Notably, the HPC domain lags slightly behind in maintaining style conventions for both short and long prompts, an observation discussed later in this section.

Figure 8 illustrates that the LLMs generally perform better across all prompt lengths in refactoring checks. In the ML domain, performance is particularly strong for medium and long prompts, whereas shorter prompts yield weaker results. Conversely, in the data processing domain, LLMs perform best with short prompts. However, as prompt length increases, performance becomes more variable and inconsistent, suggesting a general lack of stability in code quality across different prompt configurations.

As shown in Table 4, `Gemini` compensates for its lower Maintainability Index performance by achieving the highest results in the convention metric, scoring 18% above the average and ranking second in refactoring checks. The quality of short-prompt solutions is generally poor in terms of adherence to conventions, with average scores falling below half of the baseline. The lowest-performing model, `DeepSeek-Coder`, scores 73% below the baseline for short prompts in convention metrics but excels in refactoring

checks, achieving the best short-prompt results and the highest overall refactoring score.

For refactoring performance (see Table 5), `Codex` is the weakest model, with an average score of 0.64 (i.e., 17% below other LLMs and 31% below the baseline). No LLM surpasses the baseline on either metric. These findings suggest that, while LLMs can theoretically balance factors (such as complexity, code density, and comment ratios (see Section VI-B1)), they often struggle to apply these principles consistently within specific programming languages due to language-dependent nuances.

The consistently higher performance in refactoring checks may also indicate a lower emphasis on style conventions, which tend to have less severe implications than other coding issues. This pattern is likely influenced by the open-source data on which many LLMs are trained. Given that LLMs *do not consistently manage style conventions or reduce the need for refactoring*, developer oversight remains essential to ensure adherence to language-specific best practices.

TABLE 4. Style convention violations by LLMs.

LLMs	Short	Medium	Long	Average	Baseline
CodeLLama	0.57	0.75	0.74	0.69	0.92
Codex	0.50	**0.84**	0.78	0.71	
DeepSeek-Coder	0.25	0.82	0.80	0.62	
GPT-3.5T	0.34	0.77	0.70	0.60	
GPT-4T	0.59	0.80	0.81	0.73	
Gemini	**0.69**	0.83	**0.82**	**0.78**	
MagiCoder	0.38	0.68	0.63	0.55	
OpenCodeInterpreter	0.50	0.70	0.59	0.60	
WizardCoder	0.39	0.79	0.68	0.62	
Average	0.47	0.77	0.73	0.66	

TABLE 5. Refactor messages by LLMs.

LLMs	Short	Medium	Long	Average	Baseline
CodeLLama	0.83	0.77	0.77	0.79	0.93
Codex	0.72	0.62	0.59	0.64	
DeepSeek-Coder	**0.90**	0.82	0.85	**0.86**	
GPT-3.5T	0.72	0.85	0.78	0.78	
GPT-4T	0.72	0.82	0.81	0.78	
Gemini	0.71	**0.90**	**0.87**	0.83	
MagiCoder	0.60	0.79	0.62	0.68	
OpenCodeInterpreter	0.79	0.72	**0.87**	0.80	
WizardCoder	0.72	0.77	0.80	0.76	
Average	0.75	0.78	0.77	0.77	

3) SOURCE CODE STATISTICS

Moving to source code statistics, a clear correlation is observed between increasing prompt detail and the resulting code length produced by the LLMs (see Figure 9). Medium- and long-prompt solutions are nearly twice as long as those generated from short prompts, suggesting that LLMs tend to produce more comprehensive code when provided with detailed requirements. Furthermore, as prompt length increases, so does the variability of the generated solutions: shorter prompts typically yield denser code, while longer prompts exhibit greater dispersion. This pattern indicates that more detailed prompts lead to higher variability in outputs,

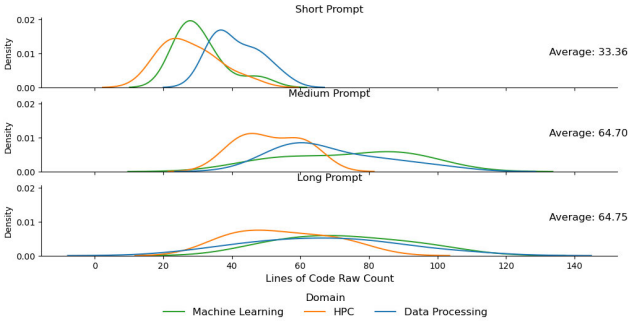


FIGURE 9. LOC across prompts and application domains.

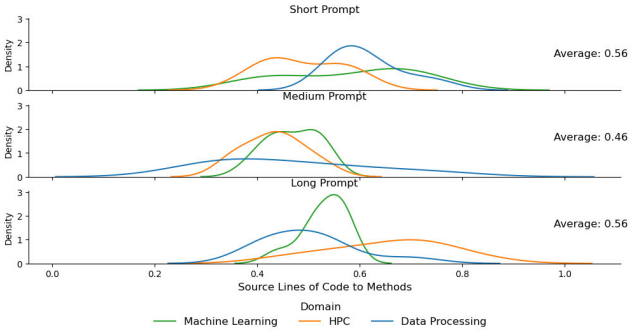


FIGURE 10. SLOC-to-methods across prompts and application domains.

reflecting the LLMs' adaptive responses to increasingly specific instructions.

The ability of LLMs to distribute logic across multiple methods exhibits considerable variability, as shown by the wide dispersion of solutions in Figure 10. Notable differences appear across domains (such as data processing with medium prompts, HPC with long prompts, and ML with short prompts) indicating domain-specific behaviors. Despite similar prompt structures, the distributions vary substantially, with medium prompts performing on average 18% lower than long prompts. Each domain demonstrates more consistent dispersion around the mean for certain prompt lengths, suggesting partial uniformity within this overall variability.

The substantial dispersion and weak correlation across domains reinforce the domain-specific differences previously noted in Python code style violations (see Section VI-B2). Moreover, the observed variability within domains highlights the challenge of designing a universal prompting methodology that effectively generalizes across diverse domains, each with distinct complexities and requirements influencing how LLMs generate and structure code.

Despite these variations, the comments-to-LOC ratio (which includes docstrings) shows greater similarity between medium and long prompts than between short ones (see Figure 11). Longer prompts, which explicitly emphasize documentation, yield higher averages than short-prompt solutions, indicating improved adherence by LLMs when such requirements are specified. Interestingly, medium prompts

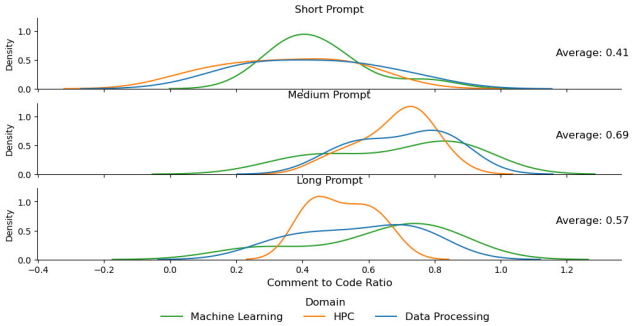


FIGURE 11. Comments-to-LOC across prompts and application domains.

TABLE 6. SLOC-to-methods ratio achieved by each LLMs.

LLMs	Short	Medium	Long	Average	Baseline
CodeLLama	0.48	**0.57**	0.58	0.54	0.66
Codex	0.55	0.41	0.50	0.49	
DeepSeek-Coder	0.57	0.43	**0.65**	0.55	
GPT-3.5T	0.49	0.50	0.55	0.51	
GPT-4T	0.59	0.46	0.51	0.52	
Gemini	0.63	0.44	0.55	0.54	
MagiCoder	0.53	0.41	**0.65**	0.52	
OpenCodeInterpreter	**0.67**	0.47	0.56	**0.57**	
WizardCoder	0.54	0.42	0.54	0.50	
Average	0.56	0.46	0.57	0.53	

TABLE 7. Comments-to-LOC ratio achieved by each LLMs.

LLMs	Short	Medium	Long	Average	Baseline
CodeLLama	0.59	0.64	0.70	0.64	0.80
Codex	0.28	0.72	0.58	0.53	
DeepSeek-Coder	0.39	**0.82**	**0.73**	0.65	
GPT-3.5T	0.19	0.65	0.48	0.44	
GPT-4T	0.55	0.74	0.70	0.66	
Gemini	**0.62**	0.79	0.71	**0.70**	
MagiCoder	0.47	0.66	0.42	0.52	
OpenCodeInterpreter	0.31	0.57	0.44	0.44	
WizardCoder	0.31	0.59	0.40	0.43	
Average	0.41	0.69	0.57	0.56	

exhibit slightly higher averages than long prompts despite comparable solution lengths (see Figure 9), suggesting that LLMs can effectively integrate comprehensive documentation when prompted without requiring excessive detail. In contrast, short prompts tend to produce more concise but less thoroughly documented solutions.

Overall, both the SLOC-to-methods ratio and the comments-to-LOC ratio display substantial dispersion, indicating limited consistency across domains and prompt lengths. However, the explicit inclusion of documentation requirements and docstring instructions in the medium and long prompts appears to have achieved the intended outcome, effectively guiding LLMs toward improved maintainability.

On average, no single LLM significantly outperforms others in terms of SLOC per method (see Table 6). Different models excel with specific prompt lengths: `OpenCodeInterpreter` performs best for short prompts, `CodeLLaMA` for medium prompts, and both `DeepSeek-Coder` and `MagiCoder` for long prompts. However, these strengths are often offset by weaker results in other configurations. Notably, `OpenCodeInterpreter` is the only model that matches the baseline's higher logic dispersion for short prompts, with `MagiCoder` and `DeepSeek-Coder` achieving comparable results.

As shown in Table 7, `Gemini` produces the best overall results for the comment-to-code ratio, achieving scores 25% higher than the LLM average, followed closely by `GPT-4T`, `DeepSeek-Coder`, and `CodeLLaMA`. However, none of the LLMs outperform the baseline, with `Gemini` averaging 12% below it. The weakest performers are `WizardCoder`, `OpenCodeInterpreter`, and `GPT-3.5T`, with averages approximately 23% lower than the overall LLM mean. Similar to the SLOC-per-method metric, `DeepSeek-Coder` produces one set of solutions with superior code documentation for specific prompts.

Across both metrics, no LLM consistently surpasses the established baseline, with only two solution sets achieving higher scores. This discrepancy likely reflects inherent differences in coding styles between human developers and LLMs. Human-generated code tends to embed widely recognized best practices more effectively, whereas LLM-generated code may be influenced by patterns present in open-source training data, which can exhibit lower maintainability. A comparable trend was previously noted in the discussion of convention violations and refactoring checks.

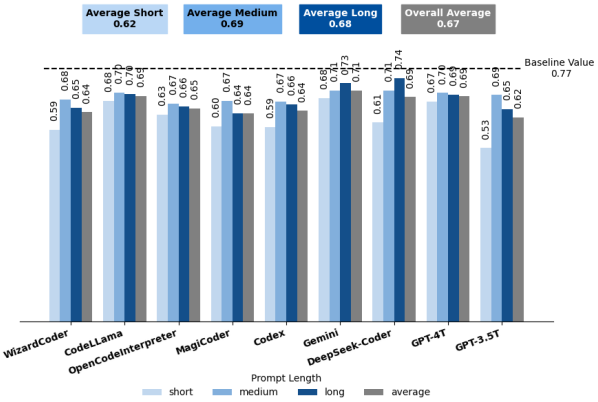


FIGURE 12. Maintainability composite score across prompts.

4) COMPOSITE SCORE

As shown in Figure 12, a clear difference is observed between the short and longer prompts. None of the LLMs match the established baseline. Overall, `Gemini` performs slightly better than the others, followed by `GPT-4T`, `DeepSeek-Coder`, and `CodeLLaMA`, all of which were previously noted for their superior maintainability scores. In terms of consistency across prompt lengths, `CodeLLaMA`, `GPT-4T`, and `OpenCodeInterpreter` perform best, with the largest deviation among them being only 6% for `OpenCodeInterpreter` between short- and medium-prompt solutions. Conversely, `GPT-3.5T` performs the worst overall, with its short-prompt solutions scoring 31% below the baseline and 15% below the average for all short prompts, followed by `Codex`, `MagiCoder`, and `WizardCoder`.

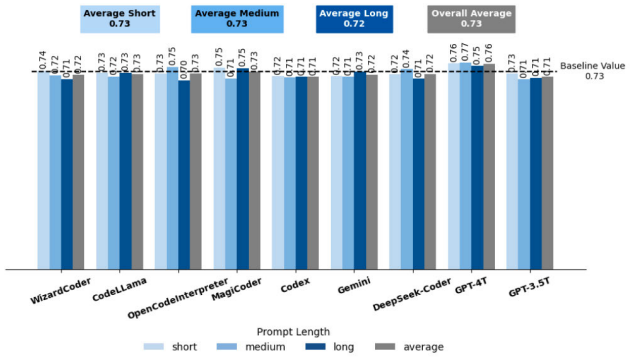


FIGURE 13. CPU usage across prompts.

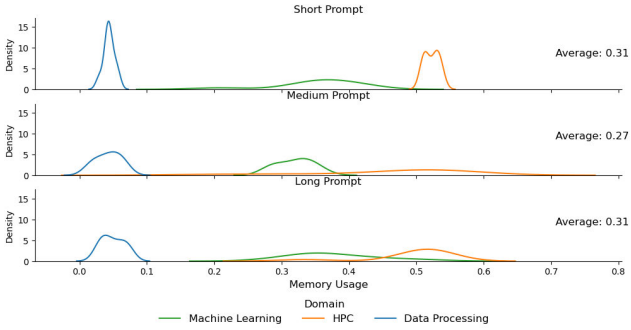


FIGURE 14. Memory usage across prompts and application domains.

C. PERFORMANCE EFFICIENCY

Preprocessing of the CPU metric involved standardizing values to a range between zero and one by dividing by 100. Memory usage was contextually normalized, making it a more reliable metric for cross-model comparison.

1) CPU USAGE

As shown in Figure 13, all LLMs exhibit comparable CPU utilization. Overall, `GPT-4T` achieves the highest average score, producing the most efficient solutions across all prompt types. However, the observed difference constitutes only a marginal improvement of approximately 4% over the baseline.

2) MEMORY USAGE

Significant variations in memory usage are observed across application domains (see Figure 14). The ML and HPC domains exhibit higher variability, whereas the data processing domain shows a strong concentration of similar results. For normalization, the maximum observed value for each algorithm was used to contextually scale memory usage, and the metric was inverted so that lower memory consumption corresponded to higher scores. Consequently, the lower values in the data processing domain indicate that most results are near the maximum, suggesting consistent memory usage across prompts and LLMs. Particularly HPC outside of short-prompt solutions display lower density, reflecting greater variability between high and low memory utilization.

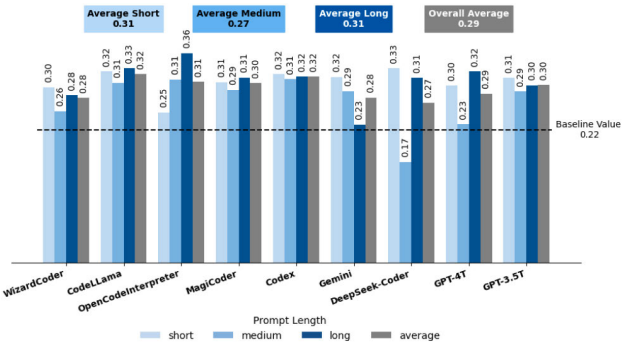


FIGURE 15. Memory usage across prompts.

As shown in Figure 15, ttDeepSeek-Coder exhibits nearly a twofold difference in memory usage between its short- and medium-prompt solutions. Similarly, `GPT-4T`, `Gemini`, and `OpenCodeInterpreter` show variations of approximately 30% between their highest and lowest performances. In contrast, models such as `Codex`, `GPT-3.5T`, `CodeLLaMA`, and `MagiCoder` demonstrate exceptional consistency across prompts, ranking among the best overall. Notably, `DeepSeek-Coder`'s medium-prompt solution failed to outperform the baseline.

Interestingly, LLMs with poorer memory efficiency tend to perform best in maintainability, whereas those with greater variability in memory usage (such as `GPT-4T`, `Gemini`, and `OpenCodeInterpreter`) achieve the highest maintainability scores. This inverse relationship underscores the trade-off between resource efficiency and structural quality, emphasizing the need to consider each model's inherent strengths and weaknesses when applying them in practice.

3) COMPOSITE SCORE

The composite score (see Figure 16) indicates that the impact of slightly higher memory usage among certain LLMs is relatively minor. The most notable under-performance is observed in `DeepSeek-Coder`'s medium-prompt solutions, which score 8% below the average for medium prompts and 4% below the baseline. Nevertheless, these variations do not lead to significant inconsistencies in overall performance.

D. RELIABILITY

Metrics such as Cyclomatic Complexity (CC) and Halstead's Number of Delivered Bugs (Bugs) offer insights into the overall reliability of the source code. In addition, Python-specific metrics obtained through `Pylint` provide warnings and error messages that highlight potential issues. If these issues remain unaddressed, the resulting code may exhibit unexpected behavior, thereby compromising operational reliability.

1) CYCLOMATIC COMPLEXITY AND BUGS

LLMs generally produce consistent results for the number of delivered bugs across prompts within the same domain (see Figure 17). However, notable differences remain between

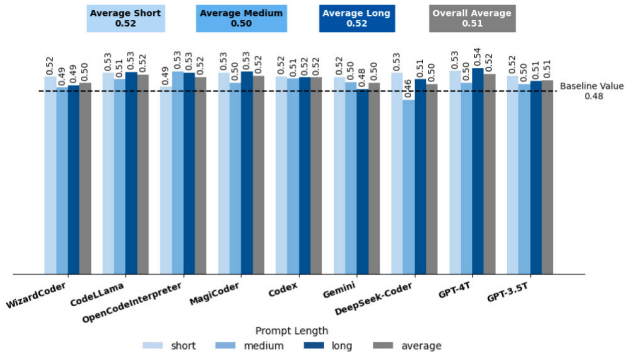


FIGURE 16. Performance efficiency-based composite score across prompts.

domains. For instance, certain algorithms in data processing tend to be more bug-prone due to their complex logic or specific data-handling procedures. Since the number of delivered bugs is derived from operands and operators in conjunction with the precise algorithmic specifications, such consistency in scoring across different prompts is to be expected.

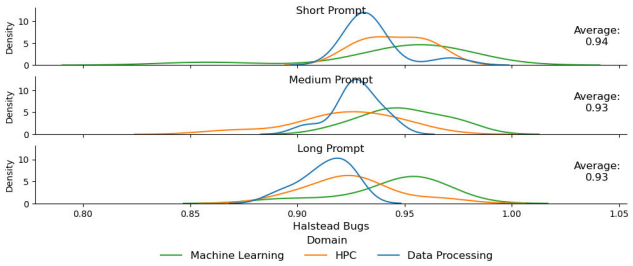


FIGURE 17. Halstead bugs across prompts and application domains.

The LLMs exhibit lower consistency across prompts when evaluating complexity per method, as shown in Figure 18. Solutions tend to display greater similarity across domains within the same prompt length than across different prompts within the same domain. This pattern suggests that LLMs maintain a consistent complexity density within prompts e.g., concentrating most logic and decision points within a single method for short prompts, while distributing them across multiple methods in longer prompts. Given the predefined complexity of the algorithms, such distribution naturally reduces overall complexity in longer prompts.

Interestingly, despite comparable code lengths (see Section VI-B3), longer-prompt solutions perform slightly below medium-prompt ones, indicating denser complexity and lower dispersion. This may result from over-specification in longer prompts, which can cause LLMs to generate unnecessarily elaborate code, thereby increasing structural complexity.

The number of delivered bugs and Cyclomatic Complexity for each LLM are presented in Table 8 and Table 9, respectively. The observed differences in consistency appear

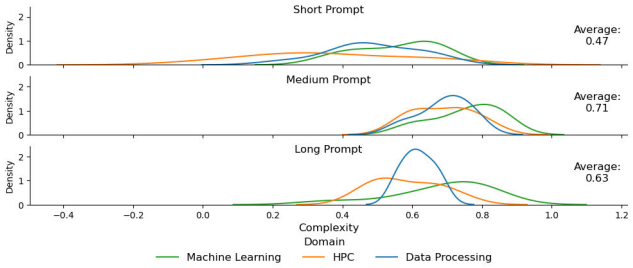


FIGURE 18. Complexity across prompts and application domains.

TABLE 8. Number of delivered bugs by each LLMs.

LLMs	Short	Medium	Long	Average	Baseline
CodeLLama	**0.97**	0.93	0.92	0.94	0.88
Codex	0.95	0.94	0.92	0.93	
DeepSeek-Coder	0.94	**0.95**	0.94	0.94	
GPT-3.5T	0.95	**0.95**	**0.95**	**0.95**	
GPT-4T	0.94	0.93	0.91	0.93	
Gemini	0.91	0.91	0.92	0.92	
MagiCoder	0.93	0.94	0.93	0.93	
OpenCodeInterpreter	0.95	0.94	0.91	0.93	
WizardCoder	0.95	**0.95**	0.94	**0.95**	
Average	0.94	0.93	0.93	0.93	

TABLE 9. Cyclomatic complexity by each LLMs.

LLMs	Short	Medium	Long	Average	Baseline
CodeLLama	0.56	0.71	0.70	0.66	0.71
Codex	0.33	0.67	0.53	0.51	
DeepSeek-Coder	0.37	0.78	0.66	0.60	
GPT-3.5T	0.36	0.71	0.48	0.57	
GPT-4T	0.65	0.78	0.64	0.69	
Gemini	**0.68**	**0.79**	**0.71**	**0.73**	
MagiCoder	0.47	0.64	0.51	0.55	
OpenCodeInterpreter	0.37	0.60	0.63	0.53	
WizardCoder	0.45	0.72	0.64	0.60	
Average	0.47	0.71	0.63	0.60	

uniform across all models, with no significant deviations in scores. Notably, `CodeLLaMA` reports the fewest delivered bugs for short-prompt solutions, although this advantage diminishes when averaged across all prompt lengths. A key observation is the baseline's under-performance, as it scores lower than all evaluated models. Since the number of bugs is derived from operands and operators, this may suggest that the baseline solutions include unnecessary operations, whereas LLM-generated code tends to adhere more closely to prompt specifications, limiting operations to what is strictly required. This consistency is notably weaker for Cyclomatic Complexity, where greater variation across methods indicates differing approaches to distributing complexity within the code, as also reflected in the domain distributions. For short prompts, `Gemini`, `GPT-4T`, and `CodeLLaMA` achieve significantly lower complexity levels (where higher scores indicate better complexity dispersion). The first two models, in particular, deviate substantially from the average, producing solutions with scores 38–44% higher than the prompt mean. These LLMs also generate lower-complexity solutions for medium prompts, resulting in overall averages that surpass those of other models.

In relation to maintainability, this suggests that these LLMs produce code aligned with best practices, ensuring that complexity is effectively distributed across methods. Consistent with findings from the domain-level analysis, medium-prompt solutions perform best on average, matching the established baseline, with several LLMs exceeding it for their medium-prompt results. In contrast, `Codex`, `OpenCodeInterpreter`, and `MagiCoder` yield the weakest outcomes, with their short-prompt solutions scoring approximately 22% below the average for all prompt lengths.

2) PYTHON SPECIFIC RELIABILITY

As shown in Table 10, Python-specific reliability metrics indicate minimal variation in warning messages, with most domains exhibiting consistent results across prompts. Notable exceptions occur in the ML domain for shorter prompts, where warnings are not reflected in other domains under similar conditions, despite these messages typically relating to coding practices rather than logic or syntax. This consistency does not extend to error metrics, which display greater variability both within and across domains (see Table 11). In particular, solutions in the HPC domain show lower quality and a higher number of errors compared to others. Results also vary significantly across prompts within the same domain. Although these metrics primarily assess adherence to correct Python programming practices, domain-specific characteristics (and possibly the limited representation of certain domains in training data) may explain the observed inconsistencies, particularly the weaker performance noted in HPC.

TABLE 10. Warning message ratios across prompts and application domains.

Application	Short Prompt	Medium Prompt	Long Prompt	Average
HPC	0.90	0.91	0.89	0.91
ML	0.74	0.86	0.90	0.85
Data Processing	0.82	0.82	0.92	0.85

TABLE 11. Error message ratios across prompts and application domains.

Application	Short Prompt	Medium Prompt	Long Prompt	Average
HPC	0.71	0.85	0.88	0.83
ML	0.99	0.99	0.87	0.96
Data Processing	0.99	0.92	0.82	0.92

As shown in Table 12 and Table 13, LLM performance on Python-specific reliability metrics exhibits notable variability. `MagiCoder` and `CodeLLaMA` tend to produce more warnings, deviating significantly from the average, whereas `GPT-4T` consistently generates the fewest warnings and errors, performing approximately 12% better than the overall mean.

Interestingly, short-prompt solutions do not necessarily correspond to lower performance. This suggests that LLMs are generally capable of avoiding major code violations, with observed deviations likely stemming from model-specific characteristics rather than prompt length alone. Notably, `GPT-3.5T` under performs in both warning and error metrics. Given the higher performance of `GPT-4T`, this discrepancy may be attributed to less refined or narrower training data.

TABLE 12. Generated warning message ratios by each LLMs.

LLMs	Short	Medium	Long	Average	Baseline
CodeLLama	0.66	0.73	0.91	0.77	0.92
Codex	0.74	0.97	0.98	0.90	
DeepSeek-Coder	0.90	0.86	0.97	0.91	
GPT-3.5T	0.82	0.74	0.86	0.81	
GPT-4T	**0.94**	0.98	**0.99**	**0.97**	
Gemini	0.86	**0.99**	0.93	0.93	
MagiCoder	0.77	0.71	0.74	0.74	
OpenCodeInterpreter	0.79	0.92	0.95	0.89	
WizardCoder	0.89	0.88	0.94	0.90	
Average	0.82	0.87	0.91	0.86	

TABLE 13. Generated errors message ratios by each LLMs.

LLMs	Short	Medium	Long	Average	Baseline
CodeLLama	**0.96**	0.89	0.76	0.87	0.97
Codex	0.88	0.95	0.96	0.93	
DeepSeek-Coder	0.79	0.96	0.86	0.87	
GPT-3.5T	0.88	0.76	0.75	0.80	
GPT-4T	0.94	**0.97**	**0.98**	**0.96**	
Gemini	0.95	0.96	0.74	0.88	
MagiCoder	0.91	0.86	0.87	0.88	
OpenCodeInterpreter	0.88	0.95	0.95	0.95	
WizardCoder	0.88	0.96	0.84	0.89	
Average	0.90	0.92	0.86	0.89	

Although the metrics exhibit variability, with a few notable under performers, most LLMs demonstrate consistently strong performance overall. This may be attributed to the higher severity of warnings and errors, which are likely less prevalent in training data compared to the style-related convention and refactoring metrics discussed in Subsection VI-B.

3) COMPOSITE SCORE

As shown in Figure 19, GPT-4T demonstrates the highest reliability, consistently outperforming the baseline across nearly all prompt lengths. It is closely followed by `Gemini`, which also maintains strong overall performance. In contrast, `MagiCoder`, `GPT-3.5T`, and `CodeLLaMA` exhibit weaker results. Short-prompt solutions were found to have approximately 10% lower reliability than medium-prompt solutions.

These findings suggest that both prompt length and model-specific characteristics influence the reliability of generated code. Certain LLMs show stronger adherence to Python coding standards, which may enhance reliability over time. While the observed differences are not substantial, incremental improvements in reliability could collectively reduce the need for extensive bug fixing in future applications.

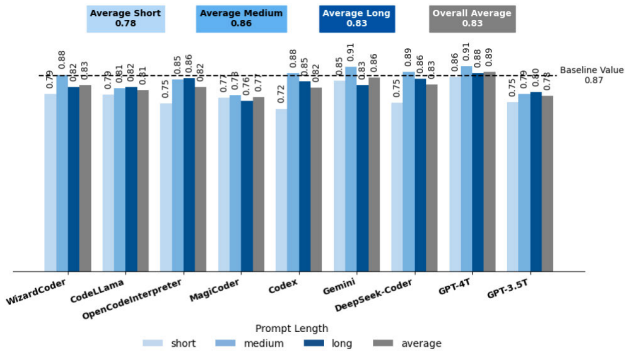


FIGURE 19. Reliability composite score across prompts.

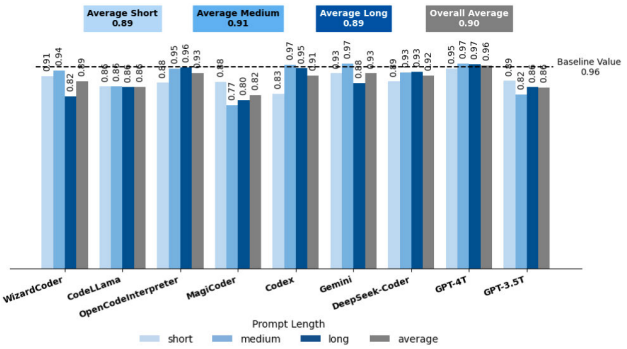


FIGURE 21. Security performance across prompts.

E. SECURITY

This section focuses exclusively on the aggregated security score to facilitate an overall assessment of compliance with security standards.

1) COMPOSITE SCORE

For security, only the composite score was used. As shown in Figure 20, LLMs demonstrate consistently high performance across all domains and prompt lengths, with no significant deviations. Although the data processing domain displays a slightly broader distribution for medium prompts, this variation is negligible when considering the x-axis range and overall performance consistency. Consequently, no notable domain-specific security vulnerabilities are indicated among the evaluated LLMs.

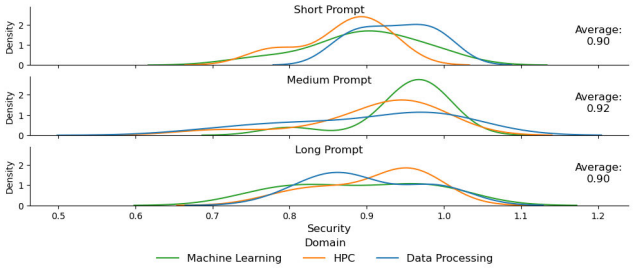


FIGURE 20. Security across prompts and application domains.

Although performance remains generally consistent, several LLMs demonstrate slightly superior results compared to others (see Figure 21), including `GPT-4T`, `OpenCodeInterpreter`, `Gemini`, `DeepSeek-Coder`, and `Codex`. Notably, `GPT-4T` surpasses the baseline across two prompt lengths, achieving an average score 7% higher than the overall mean. Conversely, `MagiCoder` performs notably worse, scoring 9% below the overall average and 15% below the baseline. Similarly, `GPT-3.5T` and `CodeLLaMA` exhibit slightly weaker performance relative to other models.

VII. DISCUSSION

A. MODEL SELECTION

This study set out to answer *To what extent can LLMs conform to the four ISO/IEC 5055:2021 categories?* Overall, the findings reveal that while modern LLMs can produce code approaching human-level quality, their performance remains uneven across categories. The LLMs struggled most with Python-specific coding standards and documentation practices, where the human baseline consistently outperformed them (refer to VI-B2 and VI-B3). These deficits were particularly evident in logic dispersion and commenting quality, areas reflecting maintainability and readability weaknesses. Reliability results showed that LLM-generated code exhibited more warnings and execution errors, influenced by domain complexity (refer to VI-D2). In contrast, performance efficiency results indicated that LLMs (especially `GPT-4T` and `Gemini`) matched or exceeded human developers, producing computationally efficient code across domains (refer to VI-C3). For security, no consistent model-wide advantage was identified. Among open-source models, `DeepSeek-Coder` and `CodeLLaMA` demonstrated strong maintainability scores, suggesting that transparency and domain-specific optimization can partially offset the advantages of proprietary models. In summary, LLMs rival humans in reliability and efficiency but still lag in domain-aware coding conventions and explanatory clarity, where human developers maintain a decisive edge.

B. PROMPT LENGTH INFLUENCE

Prompt length played a pivotal role in shaping code quality. Short prompts typically yielded concise but less compliant code, while medium-length prompts that incorporated structure and documentation requests produced the most balanced results, improving maintainability and reliability (refer to VI-B4 and VI-D3). Excessively long prompts, however, often increased code complexity without proportional quality gains. These findings align with [11] and corroborate prior evidence that strategic prompt design rather than verbosity drives optimal outcomes. Thus, medium-length prompts are recommended for professional software contexts, providing the best trade-off between clarity, quality, and reproducibility.

C. APPLICATION DOMAIN VARIATION

Cross-domain comparisons revealed that model performance varies by computational context. HPC tasks exposed more reliability and style-related issues, suggesting difficulties

with recursive and numerical precision tasks. ML algorithms benefited from medium and long prompts, achieving strong maintainability and performance efficiency, while the data processing tasks yielded the most consistent and secure results with shorter prompts. These variations indicate that prompt design and LLM selection should be tailored to domain complexity. We also noted that such discrepancies stem from training data bias. Most LLMs are overexposed to general-purpose or benchmark-style problems. Future domain-specific fine-tuning is thus essential to achieve consistent ISO 5055 compliance across specialized coding tasks.

VIII. NOTE FOR PRACTITIONERS

To translate these findings into practice, this study proposes a four step guidance to developers and organizations in selecting and using LLMs effectively. They are model selection, prompt design, application domain adaptation and code validation.

A. MODEL SELECTION

For overall high reliability, security, and performance prefer proprietary models (such as `GPT-4T` or `Gemini`). For projects prioritizing maintainability, transparency, or local deployment select open-source models (such as `DeepSeek-Coder` or `CodeLLaMA`). For instruction-tuned adaptability and educational or iterative environments consider `WizardCoder` or `OpenCodeInterpreter`.

B. PROMPT DESIGN

Medium-length prompts that specify structure, documentation, and coding standards achieved the best balance between code quality and maintainability. Short prompts are suitable for exploration but may reduce quality, while overly detailed prompts risk unnecessary code complexity.

C. APPLICATION DOMAIN ADAPTATION

In HPC domains, longer prompts improve handling of computational and data-intensive tasks. In ML, medium-to-long prompts yield the best balance between accuracy and efficiency, while in data processing tasks, shorter or moderately detailed prompts produce cleaner and more resource-efficient code.

D. VALIDATION AND QUALITY ASSURANCE

Combine static analysis tools (Pylint, Radon, Bandit, and psutil) with human review to ensure adherence to ISO 5055 categories and minimize the risk of style or logical inconsistencies.

E. ACTIONABLE TAKEAWAYS

From a practitioner's perspective, effective integration of LLMs into software workflows should follow a three-step process:

1) **LLM Selection** Choose an LLM aligned with project goals (such as security, performance, or maintainability).
2) **Prompt Design** Design structured medium-length prompts emphasizing documentation and code quality standards.
3) **Code Validation** Employ static analysis and human oversight to ensure generated code meets ISO/IEC 5055 compliance.

In conclusion, the study demonstrates that, when carefully selected and properly prompted, modern LLMs can produce code that approaches human-level quality across ISO 5055 categories. For professional software teams, the recommended workflow is an iterative cycle of prompting, evaluation, and refinement to ensure continuous improvement, maintainability, and safe integration of LLM-generated code into production systems.

IX. LIMITATIONS AND THREATS TO VALIDITY

A. LIMITATIONS

Although the study provides comprehensive quantitative results across ISO/IEC 5055 categories, the discussion of why certain models outperform others remains limited because of restricted transparency and comparability across model architectures and training corpora. Proprietary LLMs (such as `GPT-4T` and `Gemini`) disclose minimal information about parameter counts, fine-tuning strategies, or data composition, preventing a detailed causal interpretation of their superior performance. In contrast, open-source models like `DeepSeek-Coder`, `CodeLLaMA`, and `WizardCoder` offer greater methodological transparency but differ significantly in architecture size, instruction-tuning techniques, and domain coverage. As a result, performance variations observed in maintainability, reliability, and efficiency could only be partially attributed to these general factors rather than empirically validated architectural features.

Consequently, while the findings highlight which models perform best, a complete interpretive analysis of why they do so remains constrained by limited public documentation and methodological opacity inherent to most large-scale LLMs.

This study, while comprehensive, has other limitations as well:

- The selection of LLMs was limited to nine state-of-the-art models from the EvalPlus benchmark and was therefore not exhaustive.
- The evaluation focused on real-world programming challenges to capture diverse contexts in which LLMs are likely to be applied. However, it may not fully represent the complex and unpredictable nature of real-world problems that often involve broader and more dynamic requirements.
- The analysis centered on ISO 5055, employing widely used static code analysis tools to assess compliance. While this method is robust, it may not capture all nuances of code quality that a human reviewer

could identify, potentially overlooking finer aspects of maintainability, reliability, performance efficiency, and security.
- Finally, the study did not account for the dynamic learning capabilities of LLMs (such as feedback-based refinement) which may improve code quality over time.

B. THREATS TO VALIDITY

The validity and reproducibility of this study may be influenced by variations in LLM configuration, prompting strategies [11], and the accuracy of the static code analysis tools used (Pylint, Radon, Bandit, and psutil). While these tools align with ISO/IEC 5055, they provide only an approximation of code quality and may overlook deeper architectural or semantic issues. The possible inclusion of benchmark-style problems (such as LeetCode or HumanEval tasks), in LLM training data could have affected model performance, potentially inflating certain results. Hardware differences between the Apple M2 Pro CPU and Nvidia A40 GPU may also have introduced minor bias in performance efficiency measurements. Finally, the rapid evolution of LLM technologies poses a temporal limitation, as ongoing model updates could influence reproducibility and reduce the long-term applicability of the findings.

X. CONCLUSION AND FUTURE WORK

This study evaluated whether the Python code generated by nine LLMs conforms to the ISO 5055 standard. The analysis identified key strengths and weaknesses of the selected models, assessing their capabilities across three application domains using prompts of varying lengths and levels of specificity to reflect diverse real-world development contexts. The results demonstrate that, when properly prompted and configured, LLMs can produce code that meets or even exceeds the human developer baseline in terms of maintainability, reliability, performance efficiency, and security. Among the evaluated models, `GPT-4T` and `Gemini` achieved marginally superior overall performance.

In future work, the same research methodology could be applied to compiled programming languages (such as C, C++, and Fortran) to evaluate the generalizability of the results and provide a more domain-specific assessment of performance efficiency. For such analyses, instruction count during execution could replace CPU usage as a more precise performance metric. Furthermore, incorporating unit tests to evaluate the intended functionality of the generated algorithms would offer valuable insights into the LLMs' ability to interpret and follow prompt specifications. Extending this evaluation across different prompt lengths would also enable a deeper understanding of how prompt structure influences functional correctness and could guide the effective integration of LLMs into software development workflows.

AVAILABILITY OF SOURCE CODE

The source code, data, prompts and documentation captured during synthesis is available from https://github.com/RasKrebs/llms4code_code_quality.git GitHub under General Public License version 3.0 (GNU 3.0).

AUTHOR STATEMENT

Rasmus Krebs: Investigation, Methodology, Investigation, Software, Visualization, Writing-Original draft. **Somnath Mazumdar**: Conceptualization, Investigation, Methodology, Resources, Writing-Review and Editing, Visualization, Supervision.

REFERENCES

[1] *IBM Global Ai Adoption Index 2023*, IBM, Armonk, NY, USA, 2024.
[2] R. A. Poldrack, T. Lu, and G. Begus, "AI-assisted coding: Experiments with GPT-4," 2023, *arXiv:2304.13187*.
[3] D. Huang, Y. Qing, W. Shang, H. Cui, and J. M. Zhang, "Effibench: Benchmarking the efficiency of automatically generated code," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 11506–11544.
[4] L. Zhong and Z. Wang, "Can LLM replace stack overflow? A study on robustness and reliability of large language model code generation," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, Mar. 2024, pp. 21841–21849.
[5] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 21558–21572.
[6] X. Du, M. Liu, K. Wang, H. Wang, J. Liu, Y. Chen, J. Feng, C. Sha, X. Peng, and Y. Lou, "Classeval: A manually-crafted benchmark for evaluating LLMs on class-level code generation," 2023, *arXiv:2308.01861*.
[7] Z. Li et al., "Automating code review activities by large-scale pre-training," Tech. Rep., 2022. [Online]. Available: https://doi.org/10.1145/3540250.3549081
[8] M. Chen, "Evaluating large language models trained on code," 2021, *arXiv:2107.03374*.
[9] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, "Program synthesis with large language models," 2021, *arXiv:2108.07732*.
[10] G. Lin and Y. Zhang, "Sparks of artificial general recommender (AGR): Experiments with ChatGPT," *Algorithms*, vol. 16, no. 9, p. 432, Sep. 2023.
[11] A. Mastropaolo, L. Pascarella, E. Guglielmi, M. Ciniselli, S. Scalabrino, R. Oliveto, and G. Bavota, "On the robustness of code generation techniques: An empirical study on GitHub copilot," in *Proc. 45th Int. Conf. Softw. Eng.*, May 2023, pp. 2149–2160.
[12] N. Nascimento, P. Alencar, and D. Cowan, "Comparing software developers with ChatGPT: An empirical investigation," 2023, *arXiv:2305.11837*.
[13] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using large language models for automated unit test generation," *IEEE Trans. Softw. Eng.*, vol. 50, no. 1, pp. 85–105, Jan. 2024.
[14] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do users write more insecure code with AI assistants?" in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2023, pp. 2785–2799.
[15] P. Vaithilingam, T. Zhang, and E. L. Glassman, "Expectation vs. Experience: Evaluating the usability of code generation tools powered by large language models," in *Proc. CHI Conf. Human Factors Comput. Syst. Extended Abstr.*, NY, NY, USA, Apr. 2022, pp. 1–7.
[16] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of GitHub copilot's code contributions," *Commun. ACM*, vol. 68, no. 2, pp. 96–105, 2021.
[17] J. Yang, H. Jin, R. Tang, X. Han, Q. Feng, H. Jiang, S. Zhong, B. Yin, and X. Hu, "Harnessing the power of LLMs in practice: A survey on ChatGPT and beyond," *ACM Trans. Knowl. Discovery from Data*, vol. 18, no. 6, pp. 1–32, Jul. 2024.
[18] *Information Technology–software Measurement–software Quality Measurement–automated Source Code Quality Measures*, Standard ISO/IEC 5055:2021, 2021.
[19] C. Ebert and A. M. Davis, "Software principles," *IEEE Softw.*, vol. 41, no. 2, pp. 21–27, Feb. 2024.
[20] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, "Software testing with large language models: Survey, landscape, and vision," *IEEE Trans. Softw. Eng.*, vol. 50, no. 4, pp. 911–936, Apr. 2024.

[21] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, and J. M. Zhang, "Large language models for software engineering: Survey and open problems," in *Proc. IEEE/ACM Int. Conf. Softw. Eng., Future Softw. Eng.*, May 2023, pp. 31–53.
[22] S. A. Licorish, A. Bajpai, C. Arora, F. Wang, and K. Tantithamthavorn, "Comparing human and LLM generated code: The jury is still out!" 2025, *arXiv:2501.16857*.
[23] A. S. Molison, M. Moraes, G. Melo, F. Santos, and W. K. G. Assuncao, "Is LLM-generated code more maintainable & reliable than human-written code?" 2025, *arXiv:2508.00700*.
[24] Z. Manna and R. J. Waldinger, "Toward automatic program synthesis," *Commun. ACM*, vol. 14, no. 3, pp. 151–165, Mar. 1971.
[25] T. Zheng, G. Zhang, T. Shen, X. Liu, B. Y. Lin, J. Fu, W. Chen, and X. Yue, "OpenCodeInterpreter: Integrating code generation with execution and refinement," in *Proc. Findings Assoc. Comput. Linguistics ACL*. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 12834–12859. [Online]. Available: https://aclanthology.org/2024.findings-acl.762/
[26] Y. Wei, Z. Wang, J. Liu, Y. Ding, and L. Zhang, "Magicoder: Source code is all you need," 2023, *arXiv:2312.02120*.
[27] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2025, pp. 5998–6008.
[28] L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima, S. Presser, and C. Leahy, "The pile: An 800gb dataset of diverse text for language modeling," 2020, *arXiv:2101.00027*.
[29] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "Codesearchnet challenge: Evaluating the state of semantic code search," 2020, *arXiv:1909.09436*.
[30] L. Leite, C. Rocha, F. Kon, D. Milojicic, and P. Meirelles, "A survey of DevOps concepts and challenges," *ACM Comput. Surveys*, vol. 52, no. 6, pp. 1–35, Nov. 2020.
[31] G. van Rossum, B. Warsaw, and A. Coghlan. (2001). *Python Enhancement Proposals*. [Online]. Available: https://peps.python.org/pep-0008/#introduction
[32] J. Liu, C. S. Xia, Y. Wang, and L. Zhang. (2024). *Evalplus Leaderboard*. [Online]. Available: https://evalplus.github.io/leaderboard.html
[33] C. Shi, H. Yang, D. Cai, Z. Zhang, Y. Wang, Y. Yang, and W. Lam, "A thorough examination of decoding methods in the era of LLMs," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2024, pp. 8601–8629.
[34] G. Marvin, N. Hellen, D. Jjingo, and J. Nakatumba-Nabende, "Prompt engineering in large language models, "Prompt engineering in large language models," Tech. Rep., 2024, p. 387. [Online]. Available: https://doi.org/10.1007/978-981-99-7962-2_30
[35] M. Shanahan, K. McDonell, and L. Reynolds, "Role-play with large language models," *Nature*, vol. 623, pp. 493–498, Nov. 2023.
[36] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," Tech. Rep., 2023. [Online]. Available: https://dl.acm.org/doi/10.5555/3600270.3602070
[37] H. Gulabovska and Z. Porkoláb. (2019). *Survey on Static Analysis Tools of Python Programs*. [Online]. Available: https://api.semanticscholar.org/CorpusID:209149918
[38] (2003). *Pylint*. [Online]. Available: https://pylint.readthedocs.io/en/1.6.0/intro.html
[39] T. J. McCabe, "A complexity measure," *IEEE Trans. Softw. Eng.*, vol. SE-3, no. 4, pp. 22–51, Apr. 1993.
[40] K. D. Welker, "The software maintainability index revisited," *CrossTalk*, vol. 14, pp. 18–21, Jan. 2001.
[41] M. H. Halstead, *Elements of Software Science* (Operating and programming systems series). Amsterdam, The Netherlands: Elsevier, 1977.
[42] I. Sommerville, *Software Engineering*. London, U.K.: Pearson, 2015, ch. 1, pp. 1–816.
[43] D. Jurafsky and M. H. Martin, *Speech and Language Processing*, 3rd ed., Upper Saddle River, NJ, USA: Prentice-Hall, 2024.



RASMUS KREBS received the degree in data science from Copenhagen Business School, Denmark. He is currently an AI Engineer with syv.ai. His research interests focus on advanced machine learning, particularly natural language processing and the development of large language models. He is especially passionate about bridging disciplines within data science to promote interdisciplinary collaboration and innovation.



SOMNATH MAZUMDAR (Member, IEEE) received the M.Sc. degree in distributed computing from Polytech Nice Sophia Antipolis, France, and the Ph.D. degree in computing systems from the University of Siena, Italy. He is currently an Assistant Professor with Copenhagen Business School, Denmark. He has also worked at leading European research institutes and contributed to numerous large-scale European and international research projects at various levels. His research interests include high-performance and high-throughput computing, performance engineering, blockchain, and machine learning.